



Hospital Resource & Patient Management System

with Emergency Bed Allocation

Name	Register Number
Mohammed Farmanullah R	1972260040
B. Balasuriya	1911260010
Bala Ashwin R	1921260102

Object Oriented Programming in C++

Faculty: Dr. Parimala Kanaga Devan .K · Slot: D

Introduction

- Hospital resource management is absolutely essential for delivering efficient and timely patient care.
- Unexpected bed shortages can cause severe delays, particularly during critical emergencies.
- Manual tracking methods may frequently result in resource mismatch and significantly longer waiting times.
- The proposed system provides an automated, real-time bed and patient management solution built in C++.



Literature Survey (2026–2020)

Year	Paper	Contribution
2026	AI Driven Secure Bed Allocation Ecosystem	Intelligent bed allocation using AI and secure resource management.
2025	Optimizing Emergency Department Patient Flow Through Bed Allocation Strategies	Simulation based optimization for reducing waiting time.
2024	Dynamic Bed Allocation in Smart Hospitals	Improved patient flow using predictive analytics.
2022	Inpatient Discharges Forecasting for Singapore Hospitals by Machine Learning	Forecasting discharge to improve bed availability.
2021	Proactive Coordination of Inpatient Bed Management	Reduced emergency department boarding using predictive coordination.
2020	Hospital Information Management System Research	Digital management of hospital resources and patient records.

Problem Statement



Manual Constraints

Manual tracking of bed availability creates bottlenecks. It leads to difficulty in maintaining accurate and updated patient records across wards.



Emergency Delays

Dangerous delays in emergency bed allocation occur frequently due to poor coordination between varying hospital departments.



Inefficiency

Consistent resource mismatch causes inefficient utilization of hospital facilities and significantly increases overall patient waiting times.

Project Objectives

-  **Automate Tracking:** Provide real-time bed availability tracking to eliminate guesswork and manual checks.
-  **Emergency Allocation:** Instantly allocate emergency beds based on assessed patient priority and severity.
-  **Secure Records:** Maintain robust and secure patient records using structured digital storage.
-  **Reduce Waiting:** Minimize patient waiting time and significantly reduce resource misallocation.
-  **Live Updates:** Provide administrative and medical staff with real-time resource status updates continuously.

Existing System

Manual Bottlenecks

The current system relies heavily on manual bed monitoring and static resource information. This disjointed approach directly leads to delayed emergency allocation and a higher possibility of critical human errors.

Manual patient registration drastically restricts tracking capabilities and lowers efficiency.



Proposed System

- A comprehensive, C++-based Hospital Resource and Patient Management System.
- **Real-Time Tracking:** Instant bed availability updates across all hospital wings.
- **Smart Triage:** Priority-based emergency bed allocation logic ensures the most critical patients are served first.
- **Digital Workflows:** Automated patient registration paired with highly secure record management.
- **OOP Efficiency:** Optimized resource allocation utilizing advanced C++ data structures.

System Modules

Patient Core Functions

- **Patient Management:** Registration and detail updates.
- **Emergency Management:** Priority triage and assessment.
- **Record Management:** Secure data storage and retrieval.

Resource Core Functions

- **Bed Management:** Real-time occupancy tracking.
- **Resource Allocation:** Matching patients to beds.
- **Admin & Reports:** System overview and status exports.

Mod 1: Patient Management

- **Digital Registration:** Swiftly register new patients into the centralized hospital database.
- **Data Integrity:** Store comprehensive patient details and medical history accurately.
- **Dynamic Updates:** Easily update patient information as their condition or treatment progresses.
- **Record Control:** Manage holistic patient records seamlessly from a single terminal.
- **Error Reduction:** Drastically reduce manual registration errors and duplicated files.



Mod 2: Emergency Priority

Automated Triage

The system actively assesses emergency patient severity and assigns logical priority scores. Critical patients automatically receive higher priority, guaranteeing much faster emergency bed allocation.

This critical module manages incoming patients smoothly using robust priority-based logic.



Mod 3: Bed Management



Real-Time Tracking

This essential module maintains comprehensive bed information across all wards. It actively tracks both available and occupied beds, updating the system status in real-time. By displaying live bed availability dashboards, staff can instantly identify and reserve suitable beds for incoming patients.

Mod 4: Resource Allocation



Intelligent Matching

Automatically match specific patient medical requirements with the appropriate, currently available hospital resources.



Priority Logic

Allocate critical beds and resources strictly based on system-assigned priority, significantly reducing resource mismatch.



Optimization

Vastly improve the overall utilization of hospital beds and provide essential support for faster emergency response times.

Mod 5: Patient Records

- **Secure Storage:** Store all sensitive patient information securely within the local system.
- **Easy Modification:** Update patient records quickly without risking data corruption.
- **Instant Access:** Retrieve historical records instantaneously when required by doctors.
- **C++ Integration:** Utilizes advanced C++ file handling mechanisms for persistent, safe storage.
- **Confidentiality:** Architecture designed to strictly protect patient confidentiality.



Mod 6: Admin & Reports (Efficiency)



The Admin module empowers staff to view bed availability, monitor records, and generate management reports. As illustrated above, the digital C++ implementation vastly outperforms manual legacy methods in speed and accuracy.

System Use Case



SIMATS
ENGINEERING

System Actors

Patient, Hospital Staff, Emergency Staff, Administrator.

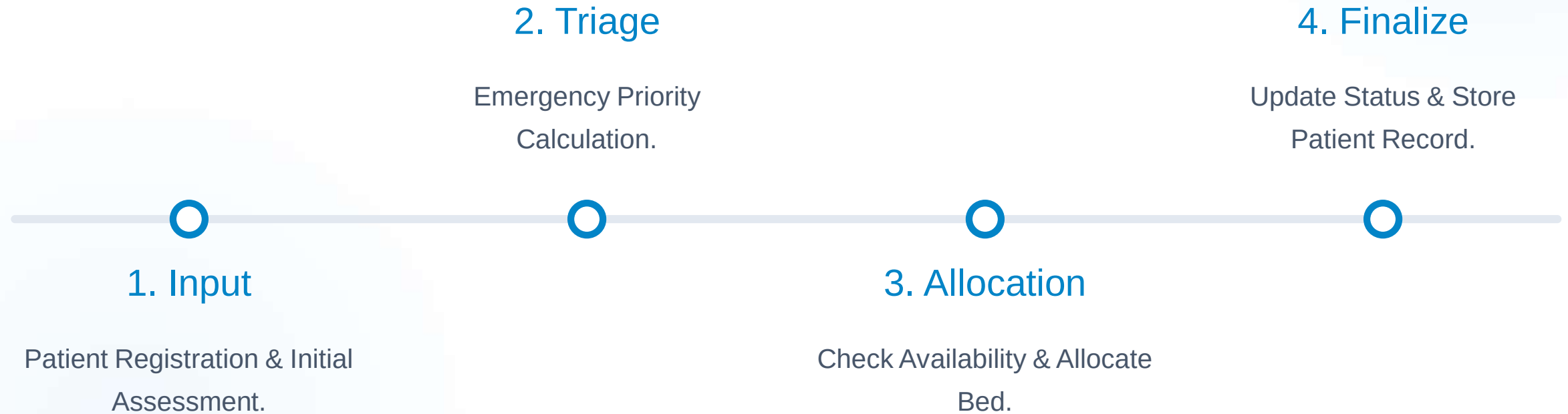
Main Use Cases

- Register Patient
- Check Bed Availability
- Assess Emergency Priority
- Allocate Bed
- Update Bed Status
- Manage Patient Records & View Reports

System Workflow



SIMATS
ENGINEERING



C++ Concepts Used



Core OOP

Utilizes fundamental Object-Oriented principles including Classes, Objects, Encapsulation, and Inheritance for modularity.



Data Structures

Implements robust Data Structures: Queues for managing patient waitlists and Arrays for tracking bed statuses efficiently.



Control & Storage

Employs precise Conditional Statements for routing logic and C++ File Handling for persistent, secure data storage.

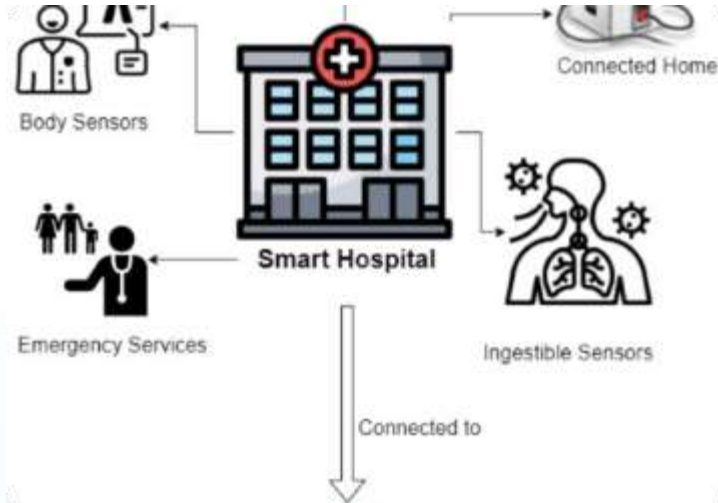
System Advantages



SIMATS
ENGINEERING

- ✓ **Rapid Action:** Faster emergency response times due to automated triage logic.
- ✓ **High Efficiency:** Better bed utilization and dramatically reduced patient waiting times.
- ✓ **Total Control:** Improved overall resource management for hospital administrators.
- ✓ **Confidentiality:** Secure, file-backed patient records protecting sensitive medical data.
- ✓ **Scalability:** Cost-effective implementation utilizing a simple and modular C++ architecture.

Future Scope & Enhancements



IoT Integration

Future integration with real hospital IoT sensors for automated, live patient monitoring.



AI Prediction

Implementing AI-based patient priority prediction to forecast emergency severity proactively.



Cloud Sync

Transition to cloud-based hospital data management and integration with global electronic health records.



Conclusion

This system significantly improves hospital resource and patient management. By implementing real-time bed tracking and priority-based allocation, it drastically reduces allocation delays and actively supports critical emergency patients. Using C++ and OOP principles provides a highly structured, reliable, and cost-effective foundation that is ready for future expansion.



References

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013.
2. Robert Lafore, *Object-Oriented Programming in C++*, 4th Edition, Sams Publishing, 2002.
3. Herbert Schildt, *C++: The Complete Reference*, 4th Edition, McGraw-Hill Education, 2014.
4. Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C++*, 2nd Edition, Silicon Press, 2007.
5. ISO — Health informatics standards and resources
6. [World Health Organization — Digital health](#)
7. [C++ Reference — cppreference.com](#)
8. [GeeksforGeeks — C++ and Data Structures](#)

```
1  #include <iostream>
2  #include <fstream>
3  #include <vector>
4  #include <queue>
5  #include <string>
6  #include <iomanip>
7  #include <algorithm>
8
9  using namespace std;
10
11  // =====
12  // PATIENT CLASS
13  // =====
14  class Patient {
15  private:
16      int patientID;
17      string name;
18      int age;
19      string gender;
20      string condition;
21      int priority;
22      int bedID;
23
24  public:
25      Patient() {
26          patientID = 0;
27          age = 0;
28          priority = 0;
29          bedID = -1;
30      }
31
32      Patient(int id, string n, int a, string g, string c, int p) {
33          patientID = id;
34          name = n;
35          age = a;
36          gender = g;
37          condition = c;
38          priority = p;
```



```
39     bedID = -1;
40 }
41
42 int getID() const {
43     return patientID;
44 }
45
46 string getName() const {
47     return name;
48 }
49
50 int getPriority() const {
51     return priority;
52 }
53
54 int getBedID() const {
55     return bedID;
56 }
57
58 void setBedID(int id) {
59     bedID = id;
60 }
61
62 void display() const {
63     cout << left
64         << setw(8) << patientID
65         << setw(20) << name
66         << setw(6) << age
67         << setw(10) << gender
68         << setw(20) << condition
69         << setw(10) << priority;
70
71     if (bedID == -1)
72         cout << setw(8) << "None";
73     else
74         cout << setw(8) << bedID;
```

```

75
76     cout << endl;
77 }
78
79 void saveToFile(ofstream &file) const {
80     file << patientID << "|"
81         << name << "|"
82         << age << "|"
83         << gender << "|"
84         << condition << "|"
85         << priority << "|"
86         << bedID << endl;
87 }
88 };
89
90 // =====
91 // BED CLASS
92 // =====
93 class Bed {
94 private:
95     int bedID;
96     string ward;
97     bool occupied;
98     int patientID;
99
100 public:
101     Bed() {
102         bedID = 0;
103         ward = "";
104         occupied = false;
105         patientID = -1;
106     }
107
108     Bed(int id, string w) {
109         bedID = id;

```

```
110     ward = w;
111     occupied = false;
112     patientID = -1;
113 }
114
115 int getBedID() const {
116     return bedID;
117 }
118
119 string getWard() const {
120     return ward;
121 }
122
123 bool isOccupied() const {
124     return occupied;
125 }
126
127 void allocateBed(int pID) {
128     occupied = true;
129     patientID = pID;
130 }
131
132 void releaseBed() {
133     occupied = false;
134     patientID = -1;
135 }
136
137 void display() const {
138     cout << left
139         << setw(10) << bedID
140         << setw(15) << ward
141         << setw(12) << (occupied ? "Occupied" : "Available");
142
143     if (occupied)
144         cout << patientID;
```

```

145         else
146             cout << "-";
147
148         cout << endl;
149     }
150 };
151
152 // =====
153 // COMPARATOR FOR EMERGENCY PRIORITY QUEUE
154 // Higher priority number = more critical
155 // =====
156 struct PatientPriority {
157     bool operator()(const Patient &a, const Patient &b) {
158         return a.getPriority() < b.getPriority();
159     }
160 };
161
162 // =====
163 // HOSPITAL MANAGEMENT SYSTEM CLASS
164 // =====
165 class HospitalSystem {
166 private:
167     vector<Patient> patients;
168     vector<Bed> beds;
169
170     priority_queue<Patient, vector<Patient>, PatientPriority> emergencyQueue;
171
172     int nextPatientID;
173
174 public:
175     HospitalSystem() {
176         nextPatientID = 1001;
177
178         // Initialize beds
179         beds.push_back(Bed(101, "General"));

```

```

178 // Initialize beds
179 beds.push_back(Bed(101, "General"));
180 beds.push_back(Bed(102, "General"));
181 beds.push_back(Bed(103, "General"));
182
183 beds.push_back(Bed(201, "Emergency"));
184 beds.push_back(Bed(202, "Emergency"));
185
186 beds.push_back(Bed(301, "ICU"));
187 beds.push_back(Bed(302, "ICU"));
188 }
189
190 // -----
191 // PATIENT REGISTRATION
192 // -----
193 void registerPatient() {
194     string name, gender, condition;
195     int age, priority;
196
197     cout << "\n===== PATIENT REGISTRATION =====\n";
198
199     cin.ignore();
200
201     cout << "Enter patient name: ";
202     getline(cin, name);
203
204     cout << "Enter age: ";
205     cin >> age;
206
207     cin.ignore();
208
209     cout << "Enter gender: ";
210     getline(cin, gender);
211
212     cout << "Enter medical condition: ";
213     getline(cin, condition);

```

```
214
215     cout << "\nEmergency Priority\n";
216     cout << "1 - Normal\n";
217     cout << "2 - Serious\n";
218     cout << "3 - Critical\n";
219     cout << "Enter priority: ";
220     cin >> priority;
221
222     if (priority < 1 || priority > 3) {
223         cout << "Invalid priority. Setting priority to 1.\n";
224         priority = 1;
225     }
226
227     Patient p(
228         nextPatientID,
229         name,
230         age,
231         gender,
232         condition,
233         priority
234     );
235
236     patients.push_back(p);
237
238     emergencyQueue.push(p);
239
240     cout << "\nPatient registered successfully!\n";
241     cout << "Patient ID: " << nextPatientID << endl;
242
243     nextPatientID++;
244 }
245
```



```

246 // -----
247 // DISPLAY PATIENTS
248 // -----
249 void displayPatients() {
250     if (patients.empty()) {
251         cout << "\nNo patient records available.\n";
252         return;
253     }
254
255     cout << "\n===== PATIENT RECORDS =====\n";
256
257     cout << left
258         << setw(8) << "ID"
259         << setw(20) << "Name"
260         << setw(6) << "Age"
261         << setw(10) << "Gender"
262         << setw(20) << "Condition"
263         << setw(10) << "Priority"
264         << setw(8) << "Bed"
265         << endl;
266
267     cout << string(82, '-') << endl;
268
269     for (const auto &p : patients) {
270         p.display();
271     }
272 }
273
274 // -----
275 // DISPLAY BEDS
276 // -----
277 void displayBeds() {
278     cout << "\n===== BED STATUS =====\n";
279
280     cout << left
281         << setw(10) << "Bed ID"
282         << setw(15) << "Ward"

```

```
283         << setw(12) << "Status"
284         << "Patient ID"
285         << endl;
286
287     cout << string(50, '-') << endl;
288
289     for (const auto &b : beds) {
290         b.display();
291     }
292 }
293
294 // -----
295 // FIND PATIENT
296 // -----
297 Patient* findPatient(int id) {
298     for (auto &p : patients) {
299         if (p.getID() == id)
300             return &p;
301     }
302
303     return nullptr;
304 }
305
306 // -----
307 // FIND AVAILABLE BED
308 // -----
309 Bed* findAvailableBed() {
310     // Emergency beds first
311     for (auto &b : beds) {
312         if (!b.isOccupied() && b.getWard() == "Emergency")
313             return &b;
314     }
315
316     // ICU beds
317     for (auto &b : beds) {
```



```

317     for (auto &b : beds) {
318         if (!b.isOccupied() && b.getWard() == "ICU")
319             return &b;
320     }
321
322     // General beds
323     for (auto &b : beds) {
324         if (!b.isOccupied() && b.getWard() == "General")
325             return &b;
326     }
327
328     return nullptr;
329 }
330
331 // -----
332 // EMERGENCY BED ALLOCATION
333 // -----
334 void allocateEmergencyBed() {
335     cout << "\n===== EMERGENCY BED ALLOCATION =====\n";
336
337     if (emergencyQueue.empty()) {
338         cout << "No patients waiting for emergency allocation.\n";
339         return;
340     }
341
342     Bed *availableBed = findAvailableBed();
343
344     if (availableBed == nullptr) {
345         cout << "No beds currently available.\n";
346         return;
347     }
348
349     Patient highestPriorityPatient = emergencyQueue.top();
350     emergencyQueue.pop();
351

```

```

352     Patient *patient =
353         findPatient(highestPriorityPatient.getID());
354
355     if (patient == nullptr) {
356         cout << "Patient record not found.\n";
357         return;
358     }
359
360     availableBed->allocateBed(patient->getID());
361
362     patient->setBedID(availableBed->getBedID());
363
364     cout << "\nBed allocated successfully!\n";
365     cout << "Patient ID : " << patient->getID() << endl;
366     cout << "Patient    : " << patient->getName() << endl;
367     cout << "Priority   : " << patient->getPriority() << endl;
368     cout << "Bed ID    : " << availableBed->getBedID() << endl;
369     cout << "Ward      : " << availableBed->getWard() << endl;
370 }
371
372 // -----
373 // RELEASE BED
374 // -----
375 void releaseBed() {
376     int bedID;
377
378     cout << "\nEnter Bed ID to release: ";
379     cin >> bedID;
380
381     for (auto &b : beds) {
382
383         if (b.getBedID() == bedID) {
384
385             if (!b.isOccupied()) {
386                 cout << "Bed is already available.\n";

```

```

386         cout << "Bed is already available.\n";
387         return;
388     }
389
390     // Find patient occupying this bed
391     for (auto &p : patients) {
392         if (p.getBedID() == bedID) {
393             p.setBedID(-1);
394             break;
395         }
396     }
397
398     b.releaseBed();
399
400     cout << "Bed " << bedID
401         << " released successfully.\n";
402
403     return;
404 }
405
406
407     cout << "Bed ID not found.\n";
408 }
409
410 // -----
411 // SEARCH PATIENT
412 // -----
413 void searchPatient() {
414     int id;
415
416     cout << "\nEnter Patient ID: ";
417     cin >> id;
418

```

```
419     Patient *patient = findPatient(id);
420
421     if (patient == nullptr) {
422         cout << "Patient not found.\n";
423         return;
424     }
425
426     cout << "\nPatient Found:\n\n";
427
428     cout << left
429         << setw(8) << "ID"
430         << setw(20) << "Name"
431         << setw(6) << "Age"
432         << setw(10) << "Gender"
433         << setw(20) << "Condition"
434         << setw(10) << "Priority"
435         << setw(8) << "Bed"
436         << endl;
437
438     cout << string(82, '-') << endl;
439
440     patient->display();
441 }
442
443 // -----
444 // SAVE RECORDS
445 // -----
446 void saveRecords() {
447     ofstream file("patient_records.txt");
448
449     if (!file) {
450         cout << "Error opening file.\n";
451         return;
452     }
453
454     for (const auto &p : patients) {
```

```
454     for (const auto &p : patients) {
455         p.saveToFile(file);
456     }
457
458     file.close();
459
460     cout << "Patient records saved successfully.\n";
461 }
462
463 // -----
464 // HOSPITAL REPORT
465 // -----
466 void generateReport() {
467     int totalBeds = beds.size();
468     int occupiedBeds = 0;
469     int availableBeds = 0;
470
471     for (const auto &b : beds) {
472         if (b.isOccupied())
473             occupiedBeds++;
474         else
475             availableBeds++;
476     }
477
478     cout << "\n===== HOSPITAL REPORT =====\n";
479
480     cout << "Total Patients      : "
481         << patients.size() << endl;
482
483     cout << "Total Beds          : "
484         << totalBeds << endl;
485
486     cout << "Occupied Beds       : "
487         << occupiedBeds << endl;
488
489     cout << "Available Beds      : "
```



```

489     cout << "Available Beds      : "
490         << availableBeds << endl;
491
492     if (totalBeds > 0) {
493         double utilization =
494             (static_cast<double>(occupiedBeds) / totalBeds) * 100;
495
496         cout << fixed << setprecision(2);
497         cout << "Bed Utilization      : "
498             << utilization << "%\n";
499     }
500
501     cout << "=====\n";
502 }
503
504 // -----
505 // MAIN MENU
506 // -----
507 void menu() {
508
509     int choice;
510
511     do {
512
513         cout << "\n\n";
514         cout << "=====\n";
515         cout << "      HOSPITAL RESOURCE & PATIENT MANAGEMENT SYSTEM\n";
516         cout << "=====\n";
517
518         cout << "1. Register Patient\n";
519         cout << "2. Display Patient Records\n";
520         cout << "3. Display Bed Status\n";
521         cout << "4. Emergency Bed Allocation\n";
522         cout << "5. Release Bed\n";
523         cout << "6. Search Patient\n";
524         cout << "7. Save Patient Records\n";

```

```
525     cout << "8. Hospital Report\n";
526     cout << "9. Exit\n";
527
528     cout << "-----\n";
529     cout << "Enter your choice: ";
530     cin >> choice;
531
532     switch (choice) {
533
534     case 1:
535         registerPatient();
536         break;
537
538     case 2:
539         displayPatients();
540         break;
541
542     case 3:
543         displayBeds();
544         break;
545
546     case 4:
547         allocateEmergencyBed();
548         break;
549
550     case 5:
551         releaseBed();
552         break;
553
554     case 6:
555         searchPatient();
556         break;
557
558     case 7:
559         saveRecords();
```



```

558         case 7:
559             saveRecords();
560             break;
561
562         case 8:
563             generateReport();
564             break;
565
566         case 9:
567             cout << "\nThank you for using the system!\n";
568             break;
569
570         default:
571             cout << "Invalid choice. Try again.\n";
572     }
573
574     } while (choice != 9);
575 }
576 };
577
578 // =====
579 // MAIN FUNCTION
580 // =====
581 int main() {
582     HospitalSystem hospital;
583
584     hospital.menu();
585
586     return 0;
587 }
588

```



SIMATS
ENGINEERING

Thank you